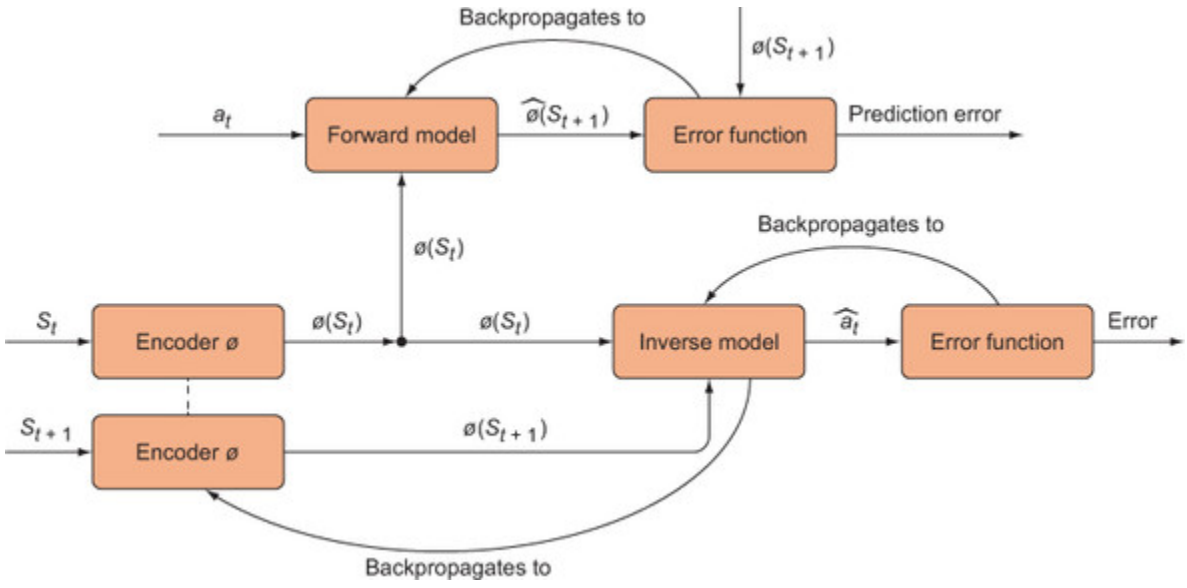
backpropagates back through to the encoder model, and the encoder model is only trained together with the inverse model. We must use PyTorch's `detach()` method to detach the forward model from the encoder so it won't backpropagate into the encoder. The purpose of the encoder is not to give us a low-dimensional input for improved performance but to learn to encode the state using a representation that only contains information relevant for predicting actions. This means that aspects of the state that are randomly fluctuating and have no impact on the agent's actions will be stripped from this encoded representation. This, in theory, should avoid the noisy TV problem.

Figure 8.10. The curiosity module. First the encoder will encode states S_t and S_{t+1} into low-dimensional vectors, $\varphi(S_t)$ and $\varphi(S_{t+1})$ respectively. These encoded states are passed to the forward and inverse models. Notice that the inverse model backpropagates to the encoded model, thereby training it through its own error. The forward model is trained by backpropagating from its own error function, but it does not backpropagate through to the encoder like the inverse model does. This ensures that the encoder learns to produce state representations that are only useful for predicting which action was taken. The black circle indicates a copy operation that copies the output from the encoder and passes the copies to the forward and inverse models.



Notice that for both the forward and inverse models we need access to the data for a full transition, i.e., we need (S_t,a_t,S_{t+1}). This is not an issue when we use an experience replay memory, as we did in chapter 3 about deep Q-learning, since the memory will store a bunch of these kinds of tuples.